

Developer A — Detection & Intelligence Playbook

Remote AI-Assisted Build Guide | Shared Rules + Detection/ML/Attribution Ownership

Role: Detection & Intelligence Owner

Purpose

A practical remote-development playbook for building the student-scale version of the Cross-Layer Threat Detection & Attribution Platform with AI assistance without breaking the architecture, duplicating work, or losing project content.

Non-negotiable principle

AI may generate implementation code, but the team owns the architecture, contracts, tests, integration, evidence, and final technical decisions.

PART A — Shared foundation you must understand

1. Project in one minute

The platform ingests security telemetry from multiple layers—identity, application, network, host and data/database activity—normalizes it, connects related entities, detects suspicious behaviour, correlates related alerts into incidents, reconstructs the attack path, and presents an explainable incident to an analyst.

```
Different security logs
↓
Collect + normalize
↓
Resolve users / IPs / devices / assets
↓
Feature engineering
↓
Rules + statistical/ML detection
↓
Alert fusion + calibration
↓
Correlation
↓
Attack graph + attribution
↓
Risk score + incident
↓
Analyst dashboard
```

Project boundary: this is a detection-and-explanation platform, not a firewall, endpoint blocker, log archive, or replacement for an identity/endpoint product.

2. Team rules — both developers must follow

- **Freeze the architecture first.** Do not change core components casually because an AI tool suggests another stack.
- **One source of truth.** The shared README, architecture document, event schema, API contract and database schema are authoritative.
- **Never invent a new payload format locally.** If a required field is missing, raise it in the shared contract instead of silently changing the shape.
- **One owner per module.** The owner can implement it, but both developers should understand the integration boundary.
- **Every module must be testable independently.** Use fixtures/mock data rather than waiting for the other person.
- **Integrate early.** Do a small end-to-end integration continuously; do not wait for the final month.
- **AI is not the architect.** Ask AI to implement against frozen contracts and existing files.
- **No blind copy-paste.** Read generated code, run tests, and verify imports, types, schema and error handling.
- **No duplicate business logic.** If normalization, risk scoring, incident creation, etc. already has an owner, do not recreate it elsewhere.
- **Document decisions.** A changed dependency, schema, endpoint or algorithm must be recorded in the project docs and communicated.

3. Remote working rule

Work as two independent module owners connected by explicit contracts. Your goal is not to share every file; your goal is to share stable interfaces and test fixtures.

```
GitHub
main
  ■■■ feature/detection-intelligence
  ■■■ feature/platform-presentation
```

Shared contract files:
docs/README.md
docs/ARCHITECTURE.md
docs/EVENT_SCHEMA.md
docs/API_CONTRACT.md

docs/DATABASE_SCHEMA.md
docs/INTEGRATION_RULES.md

4. How to use AI without losing the project

Never start an AI request with only 'build my project'. Give the model the project context, the exact module, the files it may change, the contract it must obey, and the acceptance tests.

AI prompt — mandatory base template

We are building the Cross-Layer Threat Detection & Attribution Platform.

PROJECT SOURCE OF TRUTH:

- Read and follow: docs/README.md
- Read and follow: docs/ARCHITECTURE.md
- Read and follow: docs/EVENT_SCHEMA.md
- Read and follow: docs/API_CONTRACT.md
- Read and follow: docs/DATABASE_SCHEMA.md
- Read and follow: docs/INTEGRATION_RULES.md

MY MODULE:

[describe one module only]

MY RESPONSIBILITY:

[what this module owns]

INPUT CONTRACT:

[paste/reference the exact input schema]

OUTPUT CONTRACT:

[paste/reference the exact output schema]

ALLOWED CHANGES:

[list files/directories AI may modify]

DO NOT:

- change the architecture
- change shared schemas
- rename shared fields
- create duplicate implementations
- add dependencies without approval
- modify another owner's module
- invent missing requirements

REQUIREMENTS:

[list concrete requirements]

ACCEPTANCE TESTS:

[list tests that must pass]

Before writing code:

1. Summarize the existing contract.
2. Identify dependencies.
3. State exactly which files will change.
4. Flag any conflict instead of guessing.

Then implement the smallest complete change.

AI prompt — when code already exists

Read the existing repository first. Do not rewrite working code from scratch.

Goal:

[goal]

Relevant files:

[file paths]

Constraints:

[constraints]

First explain:

- current implementation
- what must change
- what must remain unchanged
- integration impact

Then make the minimal change and provide:

- changed files
- tests added/updated
- commands to run
- any unresolved issue

AI prompt — before accepting a generated change

Review this implementation against:

1. docs/README.md
2. docs/ARCHITECTURE.md
3. docs/EVENT_SCHEMA.md
4. docs/API_CONTRACT.md
5. docs/DATABASE_SCHEMA.md
6. docs/INTEGRATION_RULES.md

Check for:

- architecture drift
- schema drift
- duplicated logic
- hidden assumptions
- security issues
- broken imports
- missing validation
- missing tests
- inconsistent naming
- unnecessary dependencies

Do not rewrite everything. Give a precise issue list first.

5. Shared contract — freeze before serious coding

The reference design recommends a canonical event schema so every source maps into one stable structure. The same idea is the key to remote AI-assisted development.

```
{
  "schema_version": "1.0",
  "event_id": "evt_001",
  "tenant_id": "tenant_001",
  "observed_at": "2026-09-28T12:00:00Z",
  "received_at": "2026-09-28T12:00:01Z",
  "layer": "identity",
  "action": "authentication.failed",
  "outcome": "failure",
  "actor": {
    "user": "user01",
    "ip": "10.0.0.5",
    "device": "device01"
  },
  "target": {
    "resource": "/login",
    "asset_id": "app01",
    "criticality": 3
  },
  "entity_id": null
}
```

Rule: the exact field names and meanings must be frozen in docs/EVENT_SCHEMA.md. The example above is a project implementation contract based on the reference document; do not add fields casually.

Contract chain

```
Canonical Event
↓
Enriched Event
↓
Feature Vector
↓
Alert Envelope
↓
Incident Object
↓
Attribution / Timeline
↓
Risk Score
↓
Dashboard API
```

Definition of done for every module

- Code is committed on the owner's branch.
- Unit/integration tests pass.
- Input/output contract is documented.
- No shared contract was changed silently.
- Fixture data is included.
- README/module notes explain how to run it.
- A second developer can run the module using the documented command.
- Integration impact is stated in the pull request.

6. GitHub workflow — low-friction remote development

```
main
  ■■■ feature/detection-intelligence
  ■■■ feature/platform-presentation
```

Typical cycle:

```
git pull origin main
git checkout -b feature/<your-module>
# implement + test
git add .
git commit -m "feat: <small change>"
git push -u origin feature/<your-module>
# open PR
# review + tests
# merge
```

- Do not develop directly on main.
- Keep commits small and meaningful.
- Do not commit secrets, API keys, local databases or generated large datasets.
- Pull/rebase from main before starting a new task.
- If a merge conflict involves a shared contract, stop and resolve it together.
- After merging a shared contract change, both developers pull immediately.

Remote daily routine

Time/Stage	Action
Start	Check main, read CURRENT_STATUS.md, choose one task.
Build	Use AI with the mandatory context prompt.
Test	Run module tests and contract tests.
Commit	Small commit with clear message.
Sync	Update CURRENT_STATUS.md and notify the other developer of interface changes.
Integration	Regularly run the end-to-end fixture scenario.

PART B — Your ownership: Detection & Intelligence

7. Your responsibility

You own the analytical brain of the system: feature engineering, detection, alert generation, correlation, attack-path reconstruction, attribution and risk scoring. Your outputs must be consumable by the platform developer through frozen contracts.

8. Your module map

Module	You own	Output
Feature Engine	Windowing, volume/failure/diversity/timing/novelty/privilege/peer features	Feature Vector
Rules Engine	Rules/threshold detections	Alert Envelope
Statistical/Unsupervised	Isolation Forest and baseline/anomaly logic as scoped	Alert Envelope
Sequence Model	One sequence model; only retain if it demonstrates value	Alert Envelope
Fusion/Calibration	Combine detector scores, calibrate and abstain	Fused Alert
Correlation	Group related alerts into incidents	Incident Object
Attribution	Attack graph, entry point/patient zero, blast radius, dwell time	Attribution Result
Risk Scoring	Time-decayed evidence score with explainable contributors	Entity/Incident Risk

9. Recommended student-scale detection scope

- Keep the rules engine: it is the highest-precision starting point.
- Keep Isolation Forest or a comparable unsupervised tabular detector.
- Keep one sequence model for ordered-action anomalies.
- Do not build six independent ML engines just to match the industrial reference.
- Use graph features/NetworkX rather than a full graph database or graph neural network.
- Use calibration and an abstention path so uncertain alerts can go to analyst review.

10. Feature engineering contract

Feature family	Examples	Purpose
Volume	request count, bytes, distinct sessions	Floods, scraping, bulk export
Failure	failure rate, consecutive failures, lockout count	Brute force / credential stuffing
Diversity	distinct endpoints, target accounts, endpoint entropy	Scanning / enumeration / spray
Timing	inter-arrival statistics, burstiness, hour-of-day deviation	Automation / beaconing / off-hours
Novelty	first-seen endpoint/country/device	Account takeover / unmanaged device
Privilege	privileged operations, role changes, token minting	Escalation / persistence
Peer-relative	deviation from role cohort	Behaviour abnormal for a peer group

Important: build one feature definition that can be reused for training and inference. Do not create one feature calculation for experiments and another for live serving.

11. Alert envelope — proposed stable shape

```
{
  "alert_id": "alert_001",
  "event_id": "evt_001",
  "entity_id": "entity_001",
  "detector": "isolation_forest",
  "technique": "credential_access",
  "score": 0.91,
  "confidence": 0.87,
  "observed_at": "...",
  "evidence": [
    {"feature": "failure_rate", "value": 0.94}
  ]
}
```

Treat this as a contract draft. Freeze the final field names in docs/API_CONTRACT.md before implementation. If the platform developer needs a field, add it through a documented contract change—not an ad-hoc response.

PART C — Detection build order

12. Build in this order

1. Create the canonical event fixtures.
2. Build deterministic feature extraction.
3. Build rules and verify known scenarios.
4. Add Isolation Forest and compare it with rules.
5. Build ordered per-entity sequences.
6. Add one sequence detector.
7. Create a common alert envelope.
8. Build fusion, calibration and abstention.
9. Build correlation into one incident object per attack scenario.
10. Build the incident graph and attribution.
11. Add explainable risk scoring.
12. Expose final outputs through the agreed API.

13. Your acceptance tests

- Same input replay produces identical features.
- Every rule has a passing and failing test case.
- Detector evaluation reports precision/recall by attack class.
- Fusion produces a reliability/calibration artefact.
- Uncertain cases can abstain instead of forcing a decision.
- A known attack scenario becomes one consolidated incident.
- Ground-truth patient zero/entry point can be checked against the test corpus.
- Risk score retains the contributing evidence so the UI can explain WHY.

14. Evaluation — do not use accuracy as the headline metric

The reference design highlights the severe class imbalance problem: a detector can have high accuracy and still generate an unusable number of false positives. Report precision/recall per attack class, alerts per analyst/day for the project scenario, calibration error, attribution accuracy, incident consolidation ratio and detection latency.

15. Attack/test corpus

Create controlled scenarios with known ground truth. Each scenario should have normal events plus an attack sequence and expected labels/relationships. Vary timing, volume and ordering so the model does not simply learn one clean simulator pattern.

```
data/  
  corpus/  
    scenario_01_bruteforce/  
    scenario_02_account_takeover/  
    scenario_03_lateral_movement/  
    scenario_04_exfiltration/  
  fixtures/  
    canonical_events.json  
    expected_alerts.json  
    expected_incidents.json  
    ground_truth.json
```

PART D — AI prompts for your module

16. Feature-engineering prompt

Use the existing repository and read all project-context files first.

Task: implement the feature-engineering module.

Follow exactly:

- docs/README.md
- docs/ARCHITECTURE.md
- docs/EVENT_SCHEMA.md
- docs/DETECTION_CONTRACT.md

Do not change the event schema.

Do not create a second feature format.

Do not add dependencies without approval.

Requirements:

- deterministic features
- configurable time windows
- unit tests
- fixture-based test
- same input => same feature output

Before coding, show:

1. input fields used
2. output fields
3. files to change
4. tests to add

Then implement only this module.

17. Detection prompt

Implement [RULE / ISOLATION FOREST / SEQUENCE] detection.

Read the shared project context first.

The detector must consume the existing Feature Vector contract.

It must emit the existing Alert Envelope.

Do not change shared schemas.

Include:

- training/inference separation where applicable
- deterministic preprocessing
- model/rule version
- test fixture
- precision/recall evaluation script where applicable
- clear error handling

Do not build a new API, dashboard, database table, or duplicate feature pipeline.

18. Attribution prompt

Implement attribution using the existing Incident Object.

Requirements:

- build a per-incident directed graph
- nodes = resolved entities
- edges = observed actions with timestamps and anomaly scores
- identify the earliest anomalous root/patient-zero candidate
- calculate forward reachability/blast radius
- calculate dwell time
- preserve the path as explainable evidence

Use NetworkX/in-memory graph for the student version.

Do not introduce Neo4j or a graph neural network.

Do not change the incident schema without proposing a contract update.

PART E — Handoff to the platform developer

19. What you must provide

Artifact	Must contain
Event fixtures	Valid canonical events for normal + attack scenarios
Feature fixtures	Expected feature values for selected events
Alert fixtures	Expected alerts with detector, score, confidence and evidence
Incident fixtures	Expected consolidated incidents
Attribution fixtures	Expected root/entry point, path and blast radius
API contract	Exact request/response fields consumed by the dashboard
Run guide	Exact commands to run your modules

20. Handoff message template

```

MODULE READY: <module name>
BRANCH/PR: <link>
INPUT CONTRACT: <schema/version>
OUTPUT CONTRACT: <schema/version>
FILES CHANGED: <list>
TEST COMMAND: <command>
FIXTURES: <path>
KNOWN LIMITATIONS: <list>
INTEGRATION NOTES: <what the platform developer must know>
BREAKING CHANGE: YES/NO

```

Golden rule: the platform developer should be able to integrate your module using the contract and fixtures without opening your internal implementation.

PART F — Your definition of done

- Module has a clear owner and stable interface.
- Tests pass locally.
- Fixture data exists.
- Evaluation numbers are reproducible.
- No architecture drift.
- No undocumented schema changes.
- No duplicate implementation of shared logic.
- Documentation is updated.
- PR is small enough to review.
- End-to-end fixture still passes after merge.

21. What you should NOT spend time on

- Six different ML models merely for feature count.
- Full graph database deployment.
- Graph neural network unless the simpler graph-feature baseline clearly fails and time remains.
- Automated containment/playbooks.
- Enterprise compliance certification.
- Large-scale production infrastructure.
- Dashboard styling that does not improve analyst understanding.